

0.1 Introduction

The reachability toolkit focuses on network survivability and connectivity probability. It computes the probability that at least one active source-to-sink route survives random component failures in a given DAG network.

For some infrastructure reliability problems, this is a sufficient performance measure. In flow-carrying network models, however, the performance measure of the system also depends on the rate of resource flow rather than just the existence of an active path. This reachability module broadly aims to answer the network query of "is the network connected?" However, in networks that also model flow, knowing the likelihood of a signal arriving does not necessarily answer the query of "how much gets through".

Consider a resource distribution network where all source-to-sink paths satisfy the reachability survival threshold. Despite the network being considered connected, deliverable throughput at target nodes can still fall well short of demand if it enters a degraded capacity state. In flow-carrying networks, computing reachability alone cannot fully distinguish between a network with reachability but low capacity and one with reachability plus plenty of spare capacity at target nodes.

Using the fixed edge capacities stored on the extracted graph object in the InputProcessing step, the network flow toolkit consists of a collection of classical max-flow-based modules intended to support analysis of deliverable throughput, bottleneck structure, and the effect of structural or parametric degradation in capacitated DAG models. Specifically, given a capacitated directed acyclic graph as input, the toolkit is designed to address the following analysis objectives:

- Q1.** What maximum throughput can the network deliver under current capacities?
- Q2.** Which cut sets form the binding bottlenecks on that throughput?
- Q3.** Which component failures or capacity perturbations most reduce deliverable flow?
- Q4.** What degradation or upgrade threshold is required to maintain a target demand level?
- Q5.** How much edge and node redundancy exists in path-disjoint terms?
- Q6.** Which components are structural single points of failure?

0.2 Mathematical Model and Network Flow Semantics

Let

$$G = (V, E)$$

be a directed acyclic graph with edge-capacity function

$$c : E \rightarrow \mathbb{R}_{\geq 0},$$

where c_{uv} is the maximum admissible flow on edge $(u, v) \in E$. When node capacities are modeled, let

$$b : V \rightarrow \mathbb{R}_{\geq 0}.$$

Here b_v denotes the maximum admissible through-flow at node v . Define the flow variable

$$f : E \rightarrow \mathbb{R}_{\geq 0},$$

with $f_{uv} := f((u, v))$ for each edge $(u, v) \in E$. For node v , define

$$\delta^+(v) := \{(v, w) \in E\}, \quad \delta^-(v) := \{(u, v) \in E\}$$

as the outgoing and incoming edge sets, respectively. For a capacitated network model G with designated source and sink nodes $s, t \in V$, the primary objective is to compute the maximum deliverable throughput from s to t , subject to edge capacities c_{uv} for $(u, v) \in E$ and, when modelled, node through-flow bounds b_v for $v \in V$.

First consider a base-case network: a single-source single-sink capacitated DAG, shown in Fig. 1. Node s is the source, node t is the sink, and intermediate nodes relay flow under capacity constraints. The network performance measure must consider and quantify deliverable throughput from s to t , not just s - t path existence.

For this base case, let $f : E \rightarrow \mathbb{R}_{\geq 0}$ and $F \in \mathbb{R}_{\geq 0}$ be decision variables, where F is the total delivered flow from s to t . The optimization model is

$$\max F$$

subject to

$$0 \leq f_{uv} \leq c_{uv}, \quad \forall (u, v) \in E, \tag{1}$$

$$\sum_{e \in \delta^-(v)} f_e - \sum_{e \in \delta^+(v)} f_e = 0, \quad \forall v \in V \setminus \{s, t\}, \tag{2}$$

$$\sum_{e \in \delta^+(s)} f_e - \sum_{e \in \delta^-(s)} f_e = F, \tag{3}$$

$$\sum_{e \in \delta^-(t)} f_e - \sum_{e \in \delta^+(t)} f_e = F. \tag{4}$$

Taking F^* to be the optimal (maximum feasible) value of F . Then if the sink node t has a required demand level D_t , the flow module quantifies service adequacy through

figures/flow_chapter/toy_siso_symbolic.pdf

Figure 1: Minimal single-source single-sink capacitated DAG network.

the throughput margin

$$\Delta_t := F^* - D_t,$$

where $\Delta_t \geq 0$ means that the network can meet demand with residual margin, and $\Delta_t < 0$ indicates a throughput shortfall of $|\Delta_t|$.

Multi-Source Multi-Sink Reduction

Let the source set be $S \subseteq V$ and the sink set be $T \subseteq V$, with $|S| \geq 1$ and $|T| \geq 1$. Figure 2a shows the original multi-source multi-sink network.

By introducing a super-source s^* and a super-sink t^* , as shown in Fig. 2b, the reduced graph is

$$V' = V \cup \{s^*, t^*\},$$

$$E' = E \cup \{(s^*, s) : s \in S\} \cup \{(t, t^*) : t \in T\}.$$

The connector capacities are set to non-binding values (implemented as Inf in the solver):

$$c_{s^*s} = +\infty \quad \forall s \in S, \quad c_{tt^*} = +\infty \quad \forall t \in T.$$

Proposition 1 (Exactness of the super-terminal reduction). *The optimal value of the*

figures/flow_chapter/msss_original.pdf

figures/flow_chapter/msss_super_reduction.pdf

(a) Original multi-source multi-sink network.

(b) Equivalent super-source/super-sink reduction.

Figure 2: Graph reduction of a multi-source multi-sink capacitated DAG to an equivalent single-source single-sink max-flow problem.

multi-source multi-sink max-flow problem on G is equal to the optimal value of the single-source single-sink max-flow problem on the reduced graph $G' = (V', E')$ with terminals s^ and t^* .*

Proof. Let f be any feasible flow for the original multi-terminal problem with total value F .

We may define an augmented flow f' on G' by retaining the values of f on all original edges, setting

$$f'_{s^*s} := \sum_{e \in \delta^+(s)} f_e - \sum_{e \in \delta^-(s)} f_e \quad \forall s \in S,$$

and

$$f'_{tt^*} := \sum_{e \in \delta^-(t)} f_e - \sum_{e \in \delta^+(t)} f_e \quad \forall t \in T.$$

Since the connector edges are guaranteed to be non-binding, then all capacity constraints from original network remain satisfied. Naturally, flow conservation holds at every original transit node, while the net outflow from s^* and the net inflow to t^* are both equal to F . Therefore f' is a feasible s^* - t^* flow of value F .

Conversely, let f' be any feasible s^* - t^* flow on G' : then, restricting f' to the original edge set E gives a flow f on G . Again, conservation on all nodes in $V \setminus (S \cup T)$ is inherited directly from the reduced problem, and the super-terminal connectors account exactly for aggregate injection at the sources and aggregate withdrawal at the sinks. Therefore f is feasible for the original multi-terminal problem and has the same total value as f' . Since feasible flows map in both directions without changing objective value, the two optimal

values are equal. $\square$

Thus, solving the reduced super-source/super-sink problem is equivalent to solving the original multi-terminal problem, which is:

$$\max F$$

subject to

$$0 \leq f_{uv} \leq c_{uv}, \quad \forall (u, v) \in E, \quad (5)$$

$$\sum_{e \in \delta^-(v)} f_e - \sum_{e \in \delta^+(v)} f_e = 0, \quad \forall v \in V \setminus (S \cup T), \quad (6)$$

$$\sum_{s \in S} \left(\sum_{e \in \delta^+(s)} f_e - \sum_{e \in \delta^-(s)} f_e \right) = F, \quad (7)$$

$$\sum_{t \in T} \left(\sum_{e \in \delta^-(t)} f_e - \sum_{e \in \delta^+(t)} f_e \right) = F. \quad (8)$$

Node-Capacitated Extension via Node Splitting

The Information Propagation Framework supports both node and edge network meta-data. For capacity analysis, this allows for modelling explicit throughput constraints on nodes themselves. As shown in Figure 3, this work adopts a node-splitting approach where each constrained node $v \in V_c$ is replaced by an input node v^- and an output node v^+ , connected by an internal edge:

$$(v^-, v^+), \quad c_{v^-v^+} = b_v.$$

The original incoming edges of v are redirected to v^- , and every original outgoing edge is emitted from v^+ .

This extends the edge-capacitated flow model with

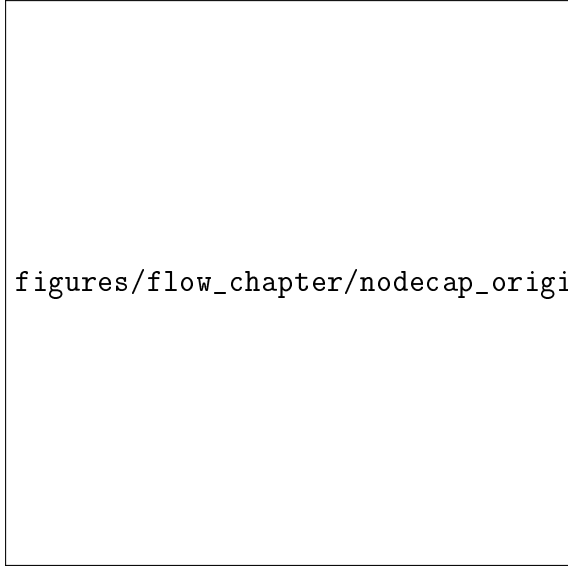
$$\sum_{e \in \delta^-(v)} f_e \leq b_v, \quad \forall v \in V_c, \quad (9)$$

$$\sum_{e \in \delta^+(v)} f_e \leq b_v, \quad \forall v \in V_c. \quad (10)$$

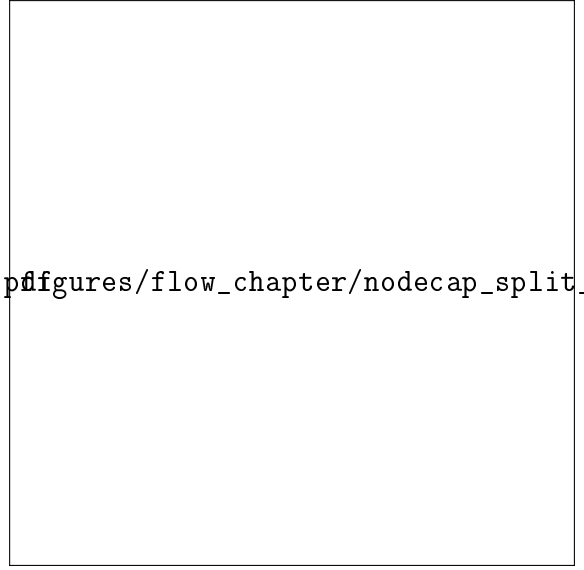
Here $V_c \subseteq V$ may include source and sink nodes when terminal throughput limits are part of the model.

The optimization model then becomes

$$\max F$$



(a) Original network with edge and node capacities.



(b) Equivalent node-split edge-capacitated network.

Figure 3: Node-splitting transformation.

subject to

$$0 \leq f_{uv} \leq c_{uv}, \quad \forall (u, v) \in E, \quad (11)$$

$$\sum_{e \in \delta^-(v)} f_e - \sum_{e \in \delta^+(v)} f_e = 0, \quad \forall v \in V \setminus \{s, t\}, \quad (12)$$

$$\sum_{e \in \delta^-(v)} f_e \leq b_v, \quad \forall v \in V_c, \quad (13)$$

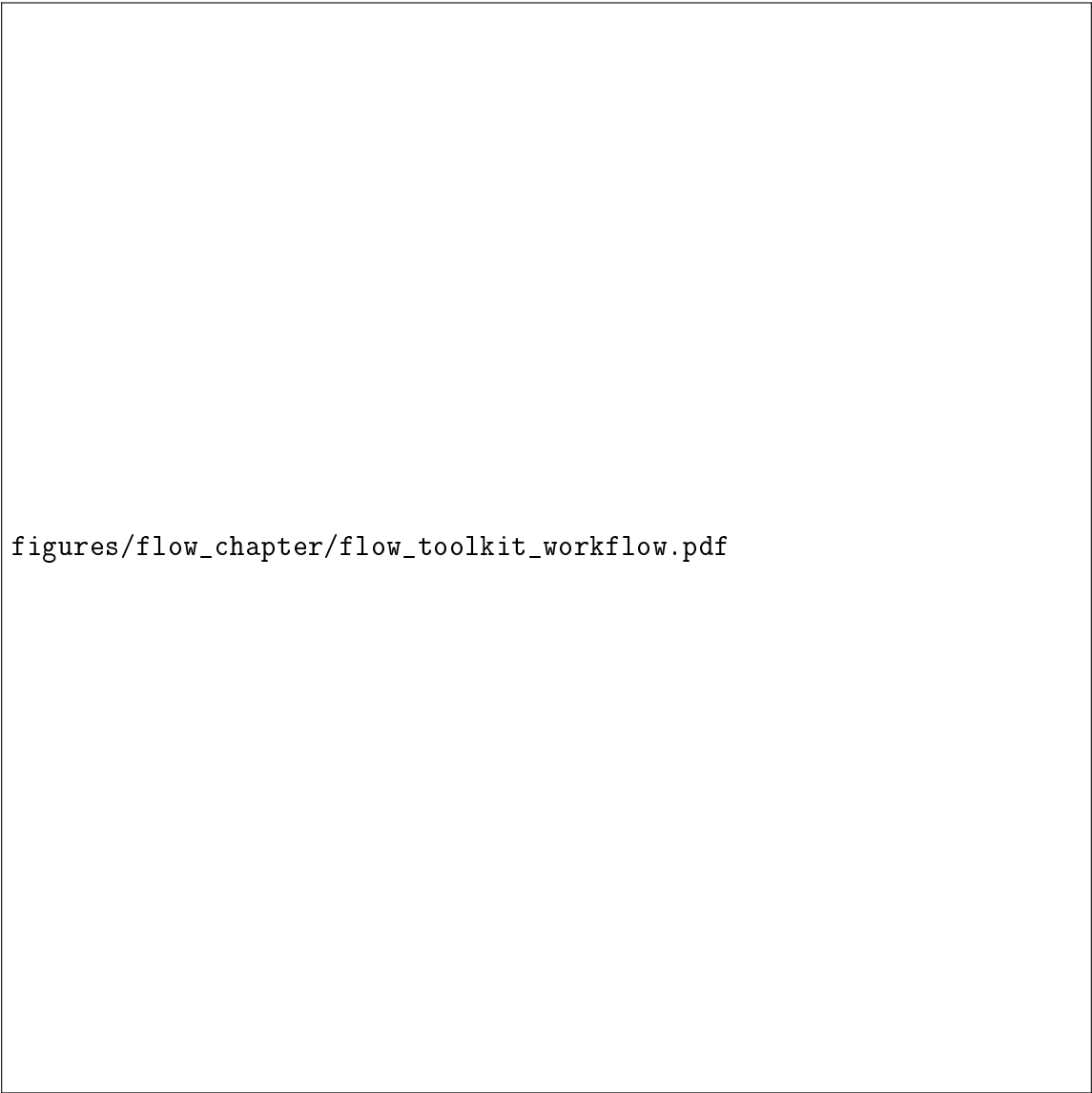
$$\sum_{e \in \delta^+(v)} f_e \leq b_v, \quad \forall v \in V_c, \quad (14)$$

$$\sum_{e \in \delta^+(s)} f_e - \sum_{e \in \delta^-(s)} f_e = F, \quad (15)$$

$$\sum_{e \in \delta^-(t)} f_e - \sum_{e \in \delta^+(t)} f_e = F. \quad (16)$$

0.3 Available Modules

Where required, the flow toolkit applies the super-terminal reduction automatically during the baseline flow solve, and the node-capacitated analysis invokes node splitting internally whenever node-capacity data are supplied. Public outputs are then remapped back to the original graph and original source/sink sets, so edge flows, sink allocations, and cut-side node sets remain interpretable in the user's native network model.



figures/flow_chapter/flow_toolkit_workflow.pdf

Figure 4: Runtime workflow for the flow-capacity toolkit with available modules

0.4 Flow Module

Motivating question: “How much flow can the network sustain, and what limits it?”

Inspired by Julia’s `Graphs.jl` and Python’s `NetworkX` flow analysis packages [?], the flow engine supports three solver variants to compute F^* for a given capacitated network G , summarised in Table 1.

Solver	Complexity	Best Fit
<code>:edmonds_karp</code>	$O(VE^2)$	Small networks
<code>:dinic</code>	$O(V^2E)$	Larger layered networks
<code>:push_relabel</code>	$O(V^2E) - O(V^3)^a$	Dense, high fan-out networks

^a Upper bound depends on node selection strategy; this solver does not enforce a highest-label invariant.

Table 1: Max-flow solver comparison.

0.4.1 Max-Flow Min-Cut Solvers

The intuition behind Ford-Folkerson’s [?] Max-Flow Min-Cut (MCMF) theorem is that given a capacitated network, the maximum permissible flow through the network is restricted by the smallest blocking bottleneck set of edges. That is, the maximum possible flow from $s \rightarrow t$ given G is equal to the sum of flow across the min-cut.

Definition 1 (Residual Graph). *Given a capacitated network $G = (V, E)$ with capacity function $c : E \rightarrow \mathbb{R}_{\geq 0}$ and a feasible flow f , the residual graph $G_f = (V, E_f)$ is defined over the same node set V with residual edge set*

$$E_f = \{(u, v) \in E : c_{uv} - f_{uv} > 0\} \cup \{(v, u) : (u, v) \in E, f_{uv} > 0\},$$

where the residual capacity on each edge is given by

$$r_{uv} = \begin{cases} c_{uv} - f_{uv} & (u, v) \in E \quad (\text{forward residual}), \\ f_{vu} & (v, u) \in E \quad (\text{backward residual}). \end{cases}$$

A forward residual edge (u, v) represents spare capacity on an original edge; a backward residual edge (v, u) represents flow on (u, v) that can be cancelled. An augmenting path P in G_f is then a directed path from s to t along which $r_{uv} > 0$ for every edge $(u, v) \in P$.

The Ford-Folkerson-based solvers work by iteratively finding and pushing flow along augmenting paths and generally follow these steps:

1. Initialise a working residual graph with zero flow.
2. Find an augmenting path P from source to sink.
3. Compute the residual (spare) capacity on all edges in P .
4. Push as much flow as possible along P , the maximum of which is limited by the edge with the least spare capacity on P .
5. Update the working graph.
6. Repeat from step 2 until no more augmenting paths remain.
7. Identify the min-cut from the reachable/non-reachable partitioning in the final residual graph.

As is evident from the algorithm steps described above, most of the computation time is spent repeating four main lookups:

- Capacity on any edge $u-v$
- Current flow on any edge $u-v$
- Outgoing edges from any node u for graph search expansion and forward residual updates
- Incoming edges to any node u for backward residual updates and inflow queries

The graph object from the input processing module already stores this information as part of the network structure and therefore does not need to be recomputed on every call. Both Ford-Folkerson-based solvers use this lookup structure but differ at step 2 in how the augmenting path is found.

Edmonds-Karp. The Edmonds-Karp solver uses a repeated breadth-first search (BFS) to find the shortest augmenting path in the residual graph. However, since each iteration requires an explicit path reconstruction via a stored parent map, the per-iteration overhead is higher than the Dinic variant, and the $O(VE^2)$ bound makes it poorly suited for larger and denser networks. It is therefore best reserved for smaller networks where simplicity and interpretability are preferred over performance.

Dinic. To avoid the one-path-at-a-time limitation of the Edmonds-Karp method, the Dinic solver alternates between level graph construction via BFS and a blocking-flow depth-first search. The level graph restricts traversal to edges $u \rightarrow v$ where $\text{level}[v] = \text{level}[u] + 1$, reducing the search space to a layered subgraph per phase. Furthermore, to avoid the $O(E)$ per-path rescanning cost of naive DFS, this variant uses a pointer cursor `ptr[u]` to track each node’s position in its adjacency list, permanently skipping exhausted edges.

The Dinic solver is the best fit for most hierarchical DAG networks and is the framework’s default when no user-specified option is selected. However, on very dense graphs with $O(V^2)$ edges and high structural redundancy, the per-phase BFS reconstruction becomes costly, and the blocking flow DFS must still handle a larger fraction of edges despite the pointer optimisation. In these cases, the Push-Relabel solver is recommended, as it operates locally without global path searches.

0.4.2 Push-Relabel Solver

Unlike the two path-based solvers, the Push-Relabel (PR) variant does not search for augmenting paths at all. This solver works by resolving excess flow locally from an initialised pre-flow. Here, instead of a residual graph per phase, a merged neighbourhood list is built once over the augmented network and shared across all push and relabel operations. The algorithm proceeds as follows:

1. Initialise $\text{height}[s^*] = |V|$, push maximum admissible flow from the super-source to its neighbours, and track $\text{excess}[v]$ as inflow minus outflow at each node.
2. While active nodes ($\text{excess}[u] > 0$) exist:
 - (a) Attempt to push flow from u to neighbour v where $\text{height}[u] = \text{height}[v] + 1$.
 - (b) If no valid push exists, relabel u by raising its height to $\min_v \text{height}[v] + 1$ over all reachable neighbours.
3. Terminate when no excess remains outside s^* and t^* .
4. Compute the min-cut by traversing the final flow state from s^* .

0.5 Network Flow Diagnostics

0.5.1 Min-Cut Utility Module

Motivating question: *“How constrained is the bottleneck, and in how many equivalent ways?”*

The flow module solvers all compute the max flow F^* by constructing a representative valid min-cut S^* in the given network G . This output by itself neither indicates nor guarantees that S^* is the only possible min-cut set in G .

This distinction matters in practice for some engineering decisions. Infrastructure systems are designed with robustness and redundancy in mind. In these networks it is more common to expect multiple structurally distinct S/T partitions of V that all have the same max flow F^* value.

While the baseline max flow value directly addresses the first analysis objective, the utility module exploits the already-generated min-cut and residual graph to build the extended partition S^{**} and compute the full space of valid min-cuts.

Let $R_T \subseteq V$ be the set of nodes reachable via backward traversal from t^* in G_f .

Definition 2 (Extended Source Partition). *The extended source-side region is*

$$S^{**} = V \setminus R_T,$$

restricted to original graph nodes. Any node outside R_T cannot reach the sink under the current flow state, so absorbing it into the source partition never increases cut capacity. Consequently $S^ \subseteq S^{**}$.*

Definition 3 (Free Zone). *Given the extended partition, the free zone is*

$$\mathcal{F} = S^{**} \setminus S^*.$$

Nodes in $\mathcal{F}$ can be assigned to either side of a minimum cut without affecting its capacity. Every minimum cut of G under f^ takes the form $S = S^* \cup R$ for some $R \subseteq \mathcal{F}$.*

The free zone size $|\mathcal{F}|$ directly determines the degeneracy of the bottleneck structure. Specifically, when $|\mathcal{F}| = 0$ the minimum cut computed from the flow solver is unique and S^* is the only valid partition. For $|\mathcal{F}| > 0$ there are exactly $2^{|\mathcal{F}|}$ equivalent minimum cuts, meaning the cut boundary is not pinned to a single partition.

Both the extended partition S^{**} and free zone $\mathcal{F}$ allow for a more complete picture of the bottleneck edge structure beyond what the representative cut alone provides. Specifically, two operationally distinct edge sets follow directly from the lattice.

The set of edges appearing in at least one minimum cut is

$$E_{\text{some}} = \{(u, v) \in E : f_{uv}^* = c_{uv}, u \in S^{**}, v \notin S^*\},$$

and the set appearing in every minimum cut is

$$E_{\text{every}} = \{(u, v) \in E : f_{uv}^* = c_{uv}, u \in S^*, v \in T^{**}\},$$

where $T^{**} = V \setminus S^{**}$, and $E_{\text{every}} \subseteq E_{\text{some}}$ holds by construction.

Min-Cut Enumerator

Given the residual lattice, the enumerator function in this utility module iterates over subsets $R \subseteq \mathcal{F}$ in binary counting order. Since each subset maps to a distinct minimum cut $S^* \cup R$, every enumerated cut is guaranteed to be valid by lattice membership. The function exposes a `cut_limit` parameter such that: if $2^{|\mathcal{F}|} \leq \text{cut_limit}$ all minimum cuts are returned and `is_complete = true`; otherwise, the first `cut_limit` cuts are returned with `is_complete = false`. This can be necessary in highly symmetric networks, where $|\mathcal{F}|$ can be large, making full enumeration intractable.

0.5.2 Structure Utility Module

Motivating question: *“Where is the network fragile, and at what granularity?”*

The network Diagnostics modules Where the Min-Cut Utilities handles the question of ‘how many valid cuts exist and which edges appear in them’, the structural module focuses on describing what the network looks like and where it is fragile. The serves as another post-processing layer on the base solver output that exposes the following helper functions.